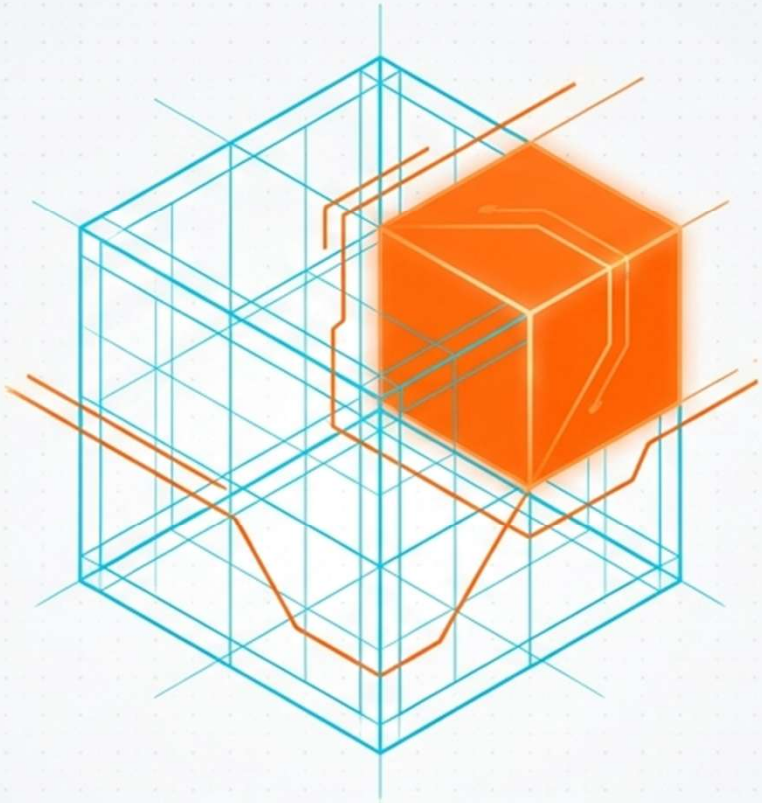


# Mastering Java: The Object-Oriented Paradigm

A visual field guide to core concepts, system architecture, and memory mechanics.

Hammad_dev

# The Paradigm Shift

## Procedural Oriented (e.g., C)

Focus: **Functions / Algorithms**
(How to do it)

Approach: **Top-Down**
(Main → Sub-Functions)

Data Flow: **Less secure**; data moves freely between functions

Reusability: **Low.** Program divided into small functions.

## Object-Oriented (e.g., Java)

Focus: **Data / Objects**
(What is involved)

Approach: **Bottom-Up**
(Build objects → Integrate them)

Data Flow: **Highly secure**; data is hidden using **Encapsulation**

Reusability: **High**, driven by **Inheritance** and **Polymorphism.**

Hammad_dev

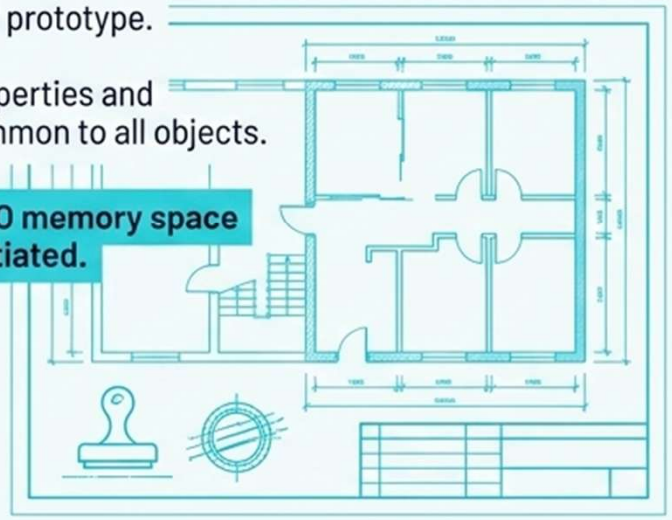
# The Blueprint & The Entity (Classes vs. Objects)

## The Class
(The Blueprint)

User-defined prototype.

Contains properties and methods common to all objects.

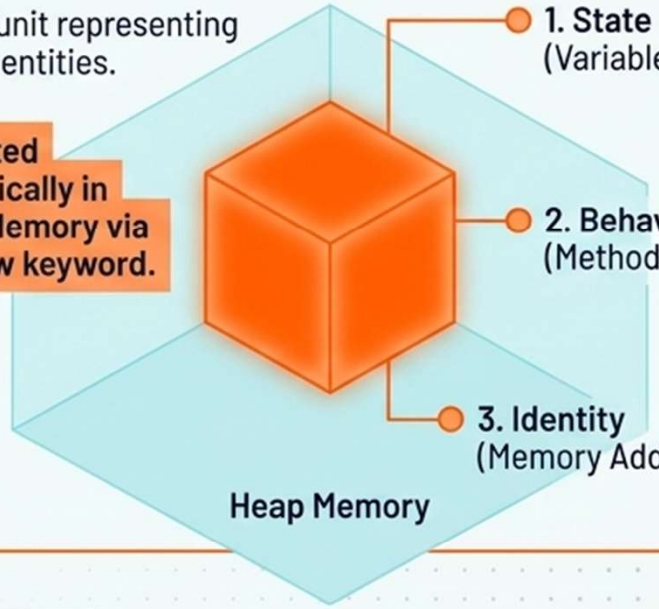
**Occupies NO memory space until instantiated.**



## The Object
(The Entity)

A basic unit representing real-life entities.

**Allocated dynamically in Heap Memory via the new keyword.**




```
Student s1 = new Student();
```

Stack reference — Heap object

Hammad_dev

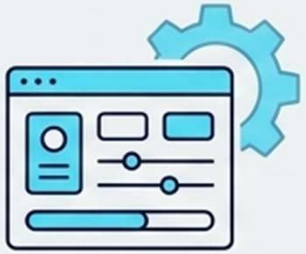
# Synthesis – The Engine of OOP

## The 4 Pillars of Object-Oriented Programming

### Encapsulation (Security)

Wrapping data and code into a single unit to enable Data Hiding.

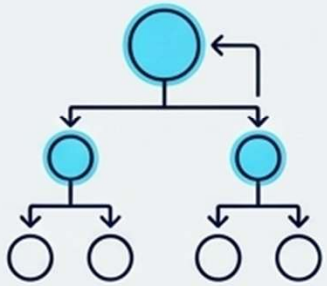
```
class Account {
  private double balance;
}
```



### Abstraction (Simplicity)

Hiding complex background implementation and showing only essential details to the user.

```
abstract void start();
// Only idea, no implementation
```



### Inheritance (Reusability)

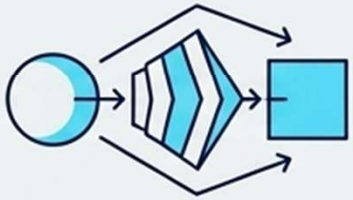
Establishing parent-child relationships so new classes can acquire properties of existing ones, eliminating redundant code.

```
class Dog extends Animal { ... }
```

### Polymorphism (Flexibility)

"One name, many forms." The ability of a variable, function, or object to take on multiple forms and perform actions in different ways.

```
void calculate(int a) {}
void calculate(double a) {}
```



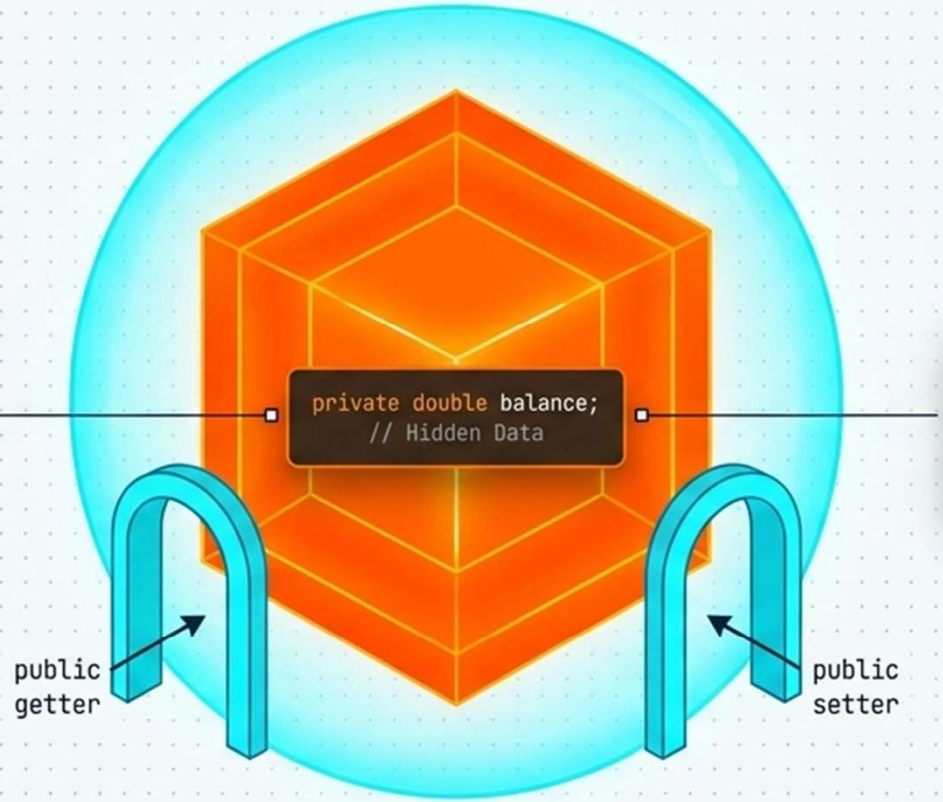
Hammad_dev

# **Encapsulation** (Packing Data)

The first pillar of OOP

## The Mechanism: Data Hiding

- Internal data cannot be accessed directly from outside the class.
- Achieved by declaring class variables as private.
- Providing public setter and getter methods to modify and view the value safely.




```
private double balance; // Hidden Data

public void deposit(double amount) {
    if (amount > 0) balance += amount;
}
```

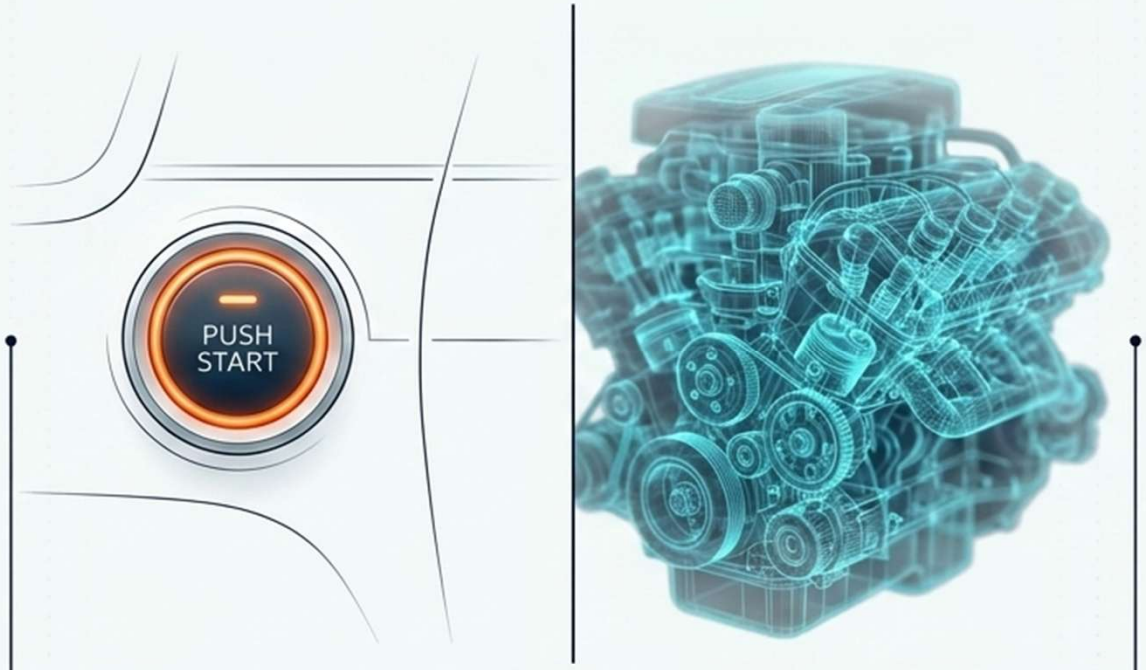
Hammad_dev

# Abstraction (Hiding Complexity)

The second pillar of OOP

### Essential Display

Displays only essential details to the user while keeping background implementation hidden.




### Hidden Implementation

Reduces programming complexity and effort.

Achieved in Java using abstract classes and interfaces.

```
abstract class Vehicle {
    abstract void start(); // Concept provided, implementation hidden
}
class Car extends Vehicle {
    void start() { System.out.println("Starts with key"); }
}
```

Hammad_dev

# Inheritance (Code Reusability)

The third pillar of OOP

## The Concept: Code Reusability

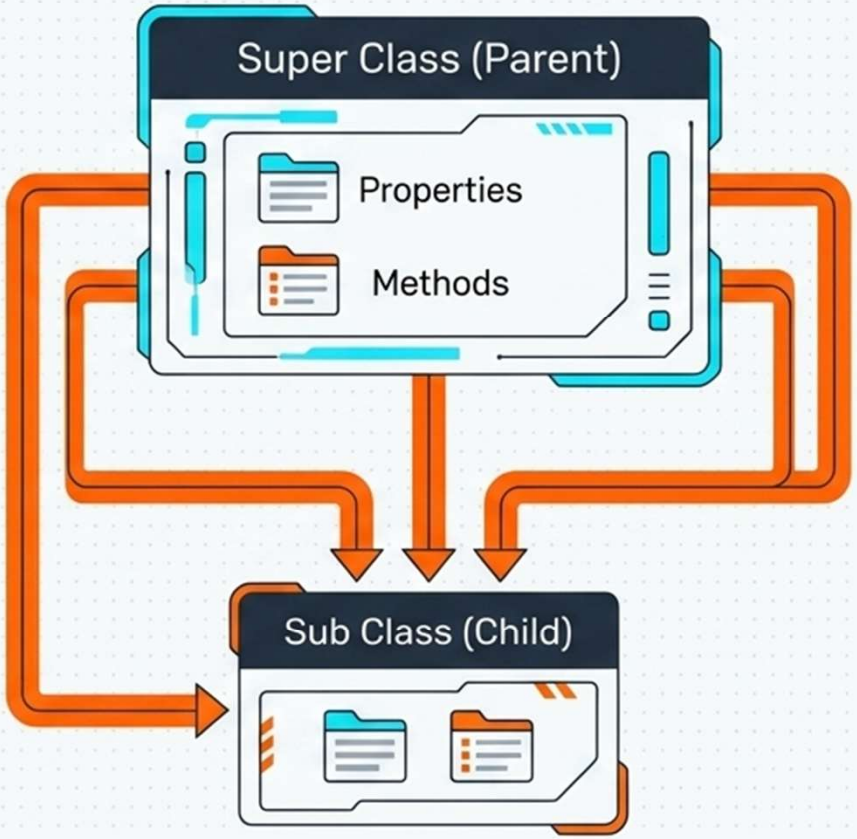
### The Mechanism:

- The process by which one class acquires the properties (fields) and behaviors (methods) of another.
- Establishes a strict Parent-Child relationship.

### Key Terminology:

- **Super Class (Parent):** The class whose features are inherited.
- **Sub Class (Child):** The class that inherits the other class.

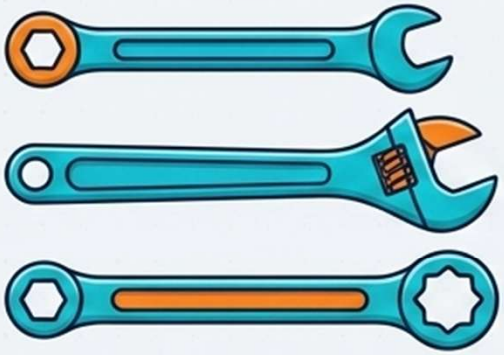
```
Code Context: Triggered using the extends
keyword.
```



# Polymorphism (One Name, Many Forms)

The final pillar of OOP

## Compile-Time: Method Overloading



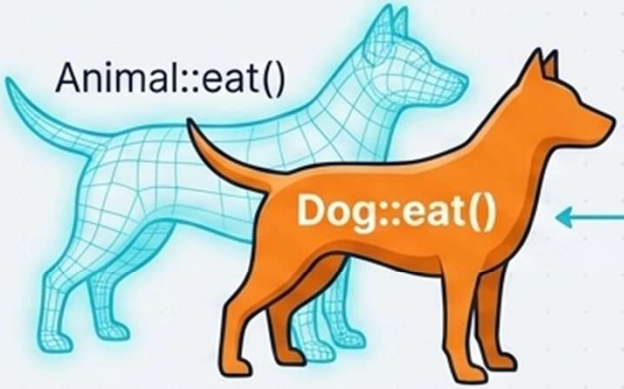
**Multiple methods** in the same class with the same name but different parameters (number or type).

Increases readability.

**Rule**: Cannot overload simply by changing the return type.

```
int add(int a, int b) { ... }
double add(double a, double b) { ... }
```

## Run-Time: Method Overriding




Providing a **specific implementation** of a method that is already provided by its parent class.

```
class Animal {
  void eat() { ... }
}

class Dog extends Animal {
  @Override
  void eat() { ... }
}
```

```
class Animal {

  @Override
  public void eat() {
    void eat()
    ... }
  }
}
```

Hammad_dev

# The Anatomy of a Java Program

Deconstructing the entry point of JVM execution.

```
public static void main(String[] args)
```

- **public** — Access control allowing the JVM (Java Virtual Machine) to execute it from anywhere.
- **static** — Allows the main method to be called without needing to create an object of the class first (class-level).
- **void** — Signifies the method returns nothing.
- **main** — The definitive starting point for JVM execution.
- **(String[] args)** — Captures command-line arguments.

```
System.out.println()
```

Calls the static `out` object from the built-in `System` class to print output to the console.

Hammad_dev

# Object Initialization & Context

Deconstructing object creation, rules, and the "this" reference.

## Constructors (Object Initializers)

**Special** method used to **initialize** objects, called **implicitly** when new is used.

**Rules:**
Must have the exact same name as the class. Must **NOT** have a **return type** (not even void).

**Types:**
**Default** (no args, sets to 0/null) vs Parameterized (takes specific args).

## Context & The `this` Keyword

A reference variable pointing to the **current class** object.

**Primary Use:**
**Resolving naming conflicts between instance variables (class fields) and local variables (parameters).**

```
this.id = id; // class field = local parameter
```

Hammad_dev

# The Building Blocks (Data Types & Casting)

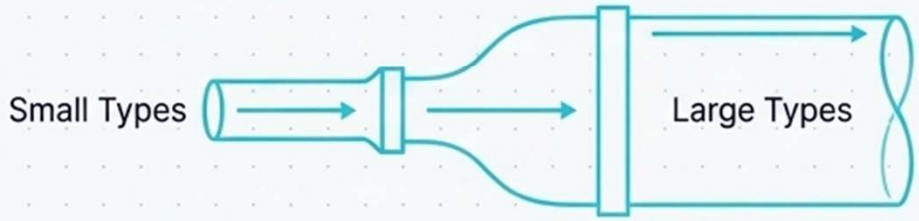
A visual field guide to Java's fundamental structures and type system rules.

## Data Types

## Type Conversion & Casting

### Casting Waterfall

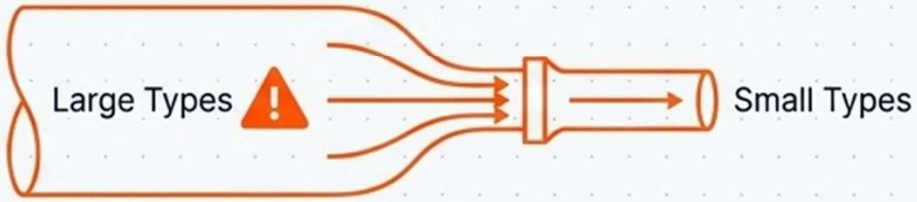
#### Widening (Implicit/Automatic)




`byte` > `short` > `char` > `int` > `long` > `float` > `double`

✓ Safe conversion. No data loss. Handled by JVM.

#### Narrowing (Explicit/Manual)




Forcing larger types into smaller sizes.
Requires parentheses (`targetType`).

⚠ **Warning:** Data loss possible (e.g., truncating decimals or overflow).

Hammad_dev

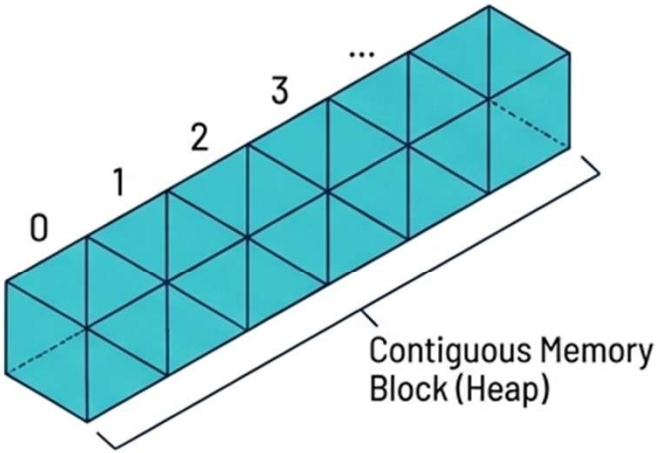
# Logic & Control Flow

| Operators | | Control Flow Statements | |
|---|---|---|---|
| Arithmetic | `+, -, *, /, %` (Modulo/Remainder) | Decision Making | `if, if-else, switch` |
| Relational | `==, !=, >, <, >=, <=` | Looping | `for, while, do-while, for-each` |
| Logical | `&&` (AND) `\|\|` (OR) `!` (NOT) | Jump | `break, continue, return` |
| Assignment | `=, +=, -=` | | |
| Ternary | `(condition) ? true_value : false_value;` (Shortest if-else) | | |

Hammad_dev

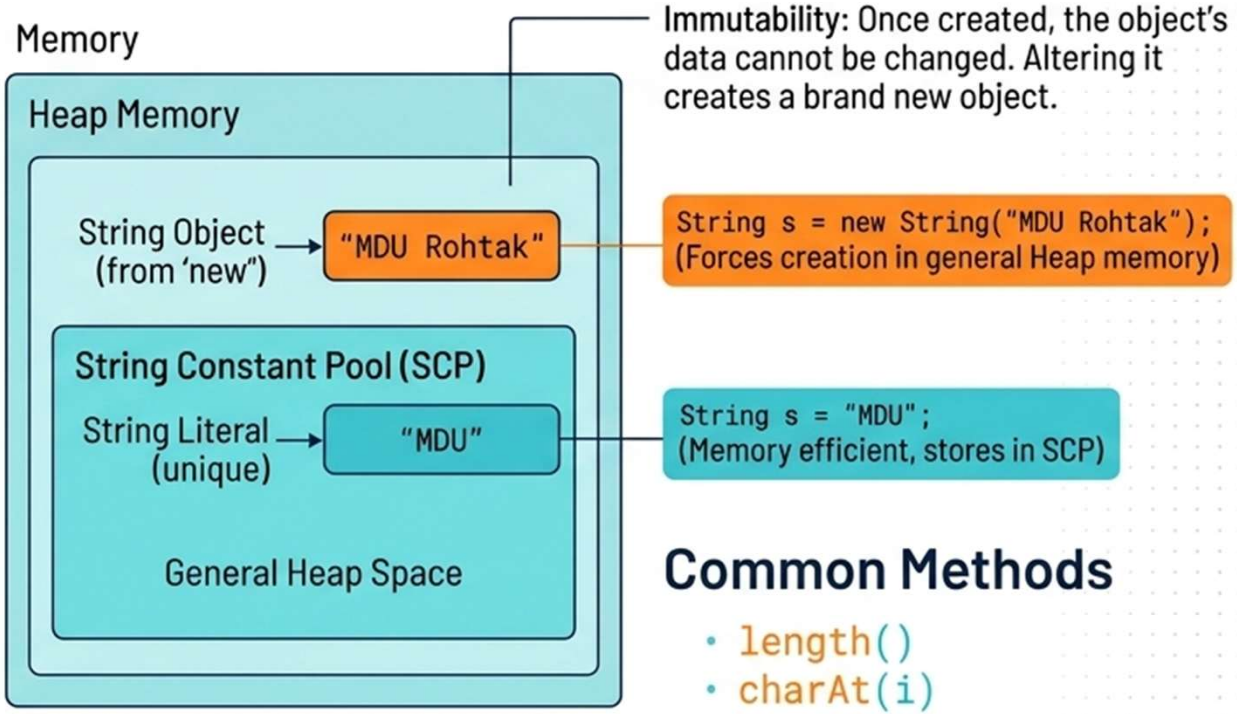
# Advanced Data Structures (Strings & Arrays)

## Arrays: Homogenous Collections




- Treated as objects, allocated dynamically on the Heap in contiguous memory.
- Safety Feature: Java checks for ArrayIndexOutOfBoundsException.

## Strings: Immutable Sequences



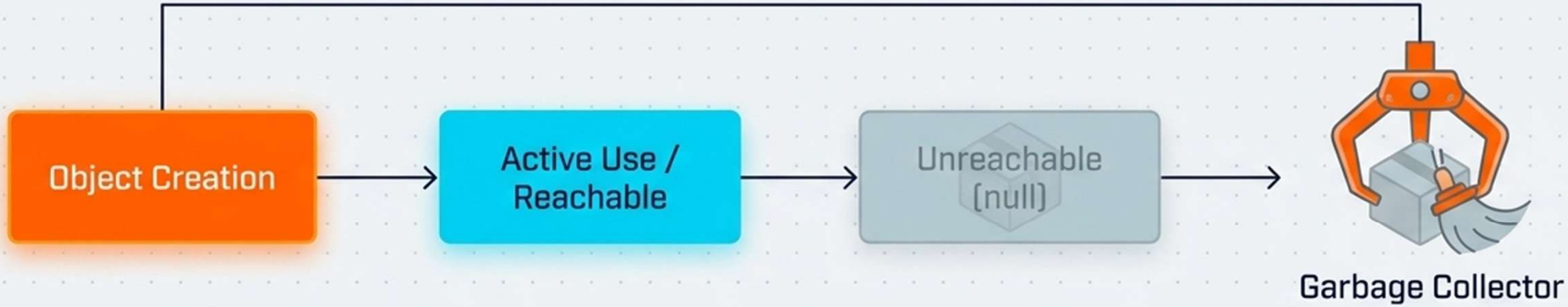

### Common Methods

- `length()`
- `charAt(i)`
- `concat()`
- `equals()`
- `substring()`

Hammad_dev

# The Final Act (Garbage Collection)

## The Object Lifecycle




### Garbage Collection (GC)

The process of automatic memory management in Java.

### How it Works

A background Daemon Thread finds and deletes objects that are no longer reachable (e.g., `obj = null;`).

### The `finalize()` Method

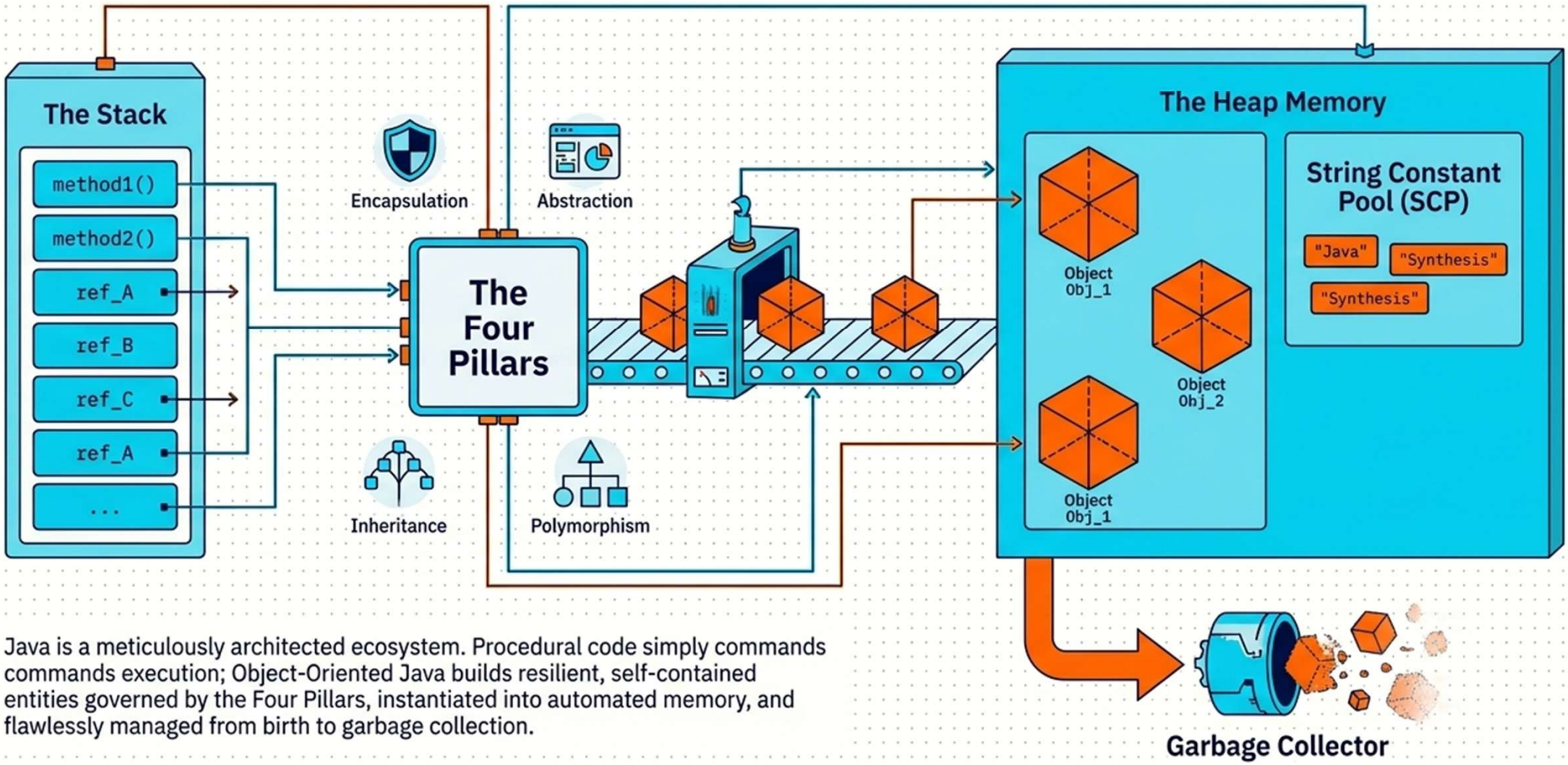
Called by GC just before destroying the object to perform cleanup activities.

### Advantage

Increases memory efficiency and prevents fatal memory leaks without manual programmer intervention.

Hammad_dev

# The Java Developer's Synthesis




Java is a meticulously architected ecosystem. Procedural code simply commands commands execution; Object-Oriented Java builds resilient, self-contained entities governed by the Four Pillars, instantiated into automated memory, and flawlessly managed from birth to garbage collection.

Hammad_dev